PromptKit

Composable, version-controlled prompts
for AI-assisted software engineering

github.com/microsoft/promptkit · MIT License · v0.1.0

Deep Dive · Architecture & Internals

The Problem with Ad-Hoc Prompts

Written once, pasted everywhere
No iteration, no improvement loop, no review process

No version control
Can't diff, blame, or revert prompt changes

No reuse
Every engineer reinvents persona, guardrails, output format

No testing
No way to verify quality or catch regressions

Inconsistent outputs
Same task → wildly different results across the team

The Fix: Treat Prompts as Code

Modular

Compose from reusable building blocks

Version-Controlled

Track changes in Git like any code

Testable

Reference-compare against known-good

Extensible

Add your own components via PR

PromptKit applies the same engineering discipline you use for software to the prompts that build it.

Architecture: The 5-Layer Composition Stack

① Persona — Who the LLM becomes

systems-engineer, security-auditor ...

② Protocols — How it reasons + guardrails

anti-hallucination, root-cause-analysis ...

③ Format — Output structure & rules

investigation-report, design-doc ...

④ Taxonomy — Domain classification (opt.)

severity tiers, risk categories

⑤ Template — The task with {{params}}

investigate-bug, review-code ...

Templates declare which components to compose via YAML frontmatter → the bootstrap engine snaps them together

Deep Dive: Personas

Layer ① — Who the LLM becomes

```
---
name: systems-engineer
description: >
  Senior systems engineer with deep
  expertise in memory management,
  concurrency, and performance.
domain:
  - systems programming
  - debugging
  - performance analysis
tone: precise, technical, methodical
---
```

Behavioral Constraints

- Reason from first principles — trace causality, never guess
- Distinguish epistemic states — label KNOWN / INFERRED / ASSUMED
- Prefer correctness over cleverness
- Explicit uncertainty — say what's missing
- No hallucination — refuse to fabricate

Available Personas

`systems-engineer`	Memory, concurrency, debugging	`security-auditor`	Vulnerability discovery, threat modeling	`software-architect`	System design, API contracts, tradeoffs
`devops-engineer`	CI/CD, release engineering, IaC	`reverse-engineer`	Spec extraction from existing code	`spl-contributor`	Library architecture, conventions

Deep Dive: Protocols

Layer ② — How the LLM reasons

Guardrails

Cross-cutting, apply to all tasks

anti-hallucination

Epistemic labeling, refusal to fabricate, source attribution

self-verification

LLM verifies its own output before finalizing

operational-constraints

Scoping, tool usage, reproducibility

Analysis

Domain/language-specific checks

memory-safety-c

Allocation, pointers, buffers

memory-safety-rust

unsafe blocks, FFI, interior mut

thread-safety

Races, deadlocks, atomics

security-vulnerability

Trust boundaries, auth, crypto

Reasoning

Systematic approaches

root-cause-analysis

Hypothesis → evidence → elimination → root cause

requirements-elicitation

Atomic, testable REQ-IDs with RFC 2119 keywords

iterative-refinement

Structure-preserving feedback cycles

Spotlight: Anti-Hallucination Protocol

The most important guardrail — prevents fabrication

Epistemic Labeling

Every claim tagged: KNOWN (directly stated), INFERRED (reasoned with explicit chain), or ASSUMED (flagged with justification).
If ASSUMED > ~30% → stop and request more context.

Refusal to Fabricate

Do NOT invent function names, API signatures, config values, file paths, or version numbers.
Use [UNKNOWN: <what is missing>] as placeholder.

Uncertainty Disclosure

Enumerate multiple interpretations rather than choosing silently. State: "Low confidence — depends on [assumption]. Verify by [action]."

Source Attribution

Indicate provenance: "per requirements doc §3.2" or "line 42 of auth.c".
Do NOT cite sources not provided.

Scope Boundaries

If question falls outside provided context, say so explicitly.
List what additional information is needed.

Spotlight: Root Cause Analysis Protocol

5-phase structured reasoning methodology

Phase 1

Symptom Characterization

Observed vs expected, timeline, blast radius, deterministic?

→

Phase 2

Hypothesis Generation

≥3 ranked hypotheses before investigating any. Include 1 non-obvious.

→

Phase 3

Evidence & Elimination

Minimal investigation per hypothesis. CONFIRMED / ELIMINATED / INCONCLUSIVE

→

Phase 4

Root Cause Identification

Distinguish root from proximate. Trace full causal chain.

→

Phase 5

Remediation

Fix root, not symptom. Secondary fixes + risk assessment.

Key rule: "If we fix only the proximate cause, will the root cause produce other failures?" → If yes, the fix is incomplete.

Example: Proximate = "null deref at line 42" · Root = "init silently fails when config missing"

Deep Dive: Output Formats

Layer ③ — Structured deliverables

```
---
name: investigation-report
type: format
produces: investigation-report
---
```

Investigation Report Structure

1. Executive Summary (2-4 sentences)
2. Problem Statement
3. Investigation Scope + limitations
4. Findings (F-NNN, severity-ordered)
5. Root Cause Analysis (causal chain)
6. Remediation Plan (priority table)
7. Prevention (code + process + tooling)
8. Open Questions
9. Revision History

All Available Formats

Format	Produces	Artifact Type
`requirements-doc`	Numbered REQ-IDs + acceptance criteria	`requirements-document`
`design-doc`	Architecture, APIs, tradeoff analysis	`design-document`
`validation-plan`	Test cases + traceability matrix	`validation-plan`
`investigation-report`	Findings, RCA, remediation	`investigation-report`
`implementation-plan`	Task breakdown + dependencies	`implementation-plan`
`agent-instructions`	.github/instructions/, CLAUDE.md, .cursorrules	`agent-instructions`
`pipeline-spec`	YAML pipelines + design rationale	`pipeline-spec`
`release-notes`	Changelog, breaking changes, upgrade path	`release-notes`
`multi-artifact`	Multiple structured files	`multi-artifact`

Deep Dive: Template Anatomy

Layer ⑤ — The task declaration

```yaml
---
name: investigate-bug
description: >
  Systematically investigate a bug.
  Apply root cause analysis and produce
  an investigation report.

persona: systems-engineer

protocols:
  - guardrails/anti-hallucination
  - guardrails/self-verification
  - guardrails/operational-constraints
  - reasoning/root-cause-analysis

format: investigation-report

params:
  problem_description: "Bug description"
  code_context: "Relevant code/logs"
  environment: "OS, runtime, build"

input_contract: null
output_contract:
```

persona:

References one persona file.
Becomes the # Identity section.

protocols:

List of category/name paths.
Each loaded as # Reasoning Protocol.

format:

References one format file.
Becomes the # Output Format section.

params:

Key-value map → {{param}} placeholders in template body get substituted.

input_contract:

What artifact this template consumes.
null = no prerequisite.

output_contract:

What artifact this template produces.
Enables pipeline chaining.

The Manifest: Source of Truth

manifest.yaml — the component registry

```yaml
# manifest.yaml
version: "0.1.0"

personas:
  - name: systems-engineer
    path: personas/systems-engineer.md

protocols:
  guardrails:
    - name: anti-hallucination
      path: protocols/guardrails/ ...
  analysis:
    - name: memory-safety-c
      path: protocols/analysis/ ...
  reasoning:
    - name: root-cause-analysis
      path: protocols/reasoning/ ...

formats:
  - name: investigation-report
    path: formats/investigation-report.md
    produces: investigation-report

templates:
  investigation:
```

Key Rules

Protocol naming duality

Templates use category paths:
  guardrails/anti-hallucination
Manifest uses short names:
  anti-hallucination
CI enforces they stay in sync.

Path resolution

Every component has a path field pointing to its .md source file. Assembly engine reads these paths.

Pipeline stages

produces/consumes fields enable the bootstrap engine to validate chaining and suggest next stages.

CI validation

tests/validate-manifest.py checks that manifest protocols match template frontmatter protocols.

The Bootstrap Flow

bootstrap.md — the meta-prompt that drives composition

1 Understand the task
Ask user what they want to accomplish

›

2 Read manifest
Discover all available components

›

3 Select template
Match task to best template

›

4 Check mode
Interactive → execute
Single-shot → file

›

5 Collect params
Gather template-specific inputs

›

6 Assemble
Compose layers in order

›

7 Output
Write prompt file or execute directly

Single-Shot Mode (default)

```
Gather params → assemble prompt → write .md file
User pastes into any LLM session

Output options:
  (a) Raw prompt file
  (b) Agent instructions (.github/instructions/,
       CLAUDE.md, .cursorrules)
```

Interactive Mode

```
Load components → execute directly in session
Reasoning-and-challenge phase before output

Template declares: mode: interactive
Used by: extend-library, interactive-design
No file written — LLM acts on the prompt live
```

Assembly Engine Internals

cli/lib/assemble.js — how the sausage is made

Identity

Load persona .md, strip frontmatter

Reasoning Protocols

Load each protocol .md, join with ---

Output Format

Load format .md, strip frontmatter

Task

Load template .md, substitute {{params}}

Key Processing Steps

stripFrontmatter()

Removes YAML --- block from each component before assembly

Strip HTML comments

Removes SPDX license headers (<!-- ... --> blocks)

loadComponent()

Reads file, strips frontmatter + comments, returns clean body

Parameter substitution

Simple {{key}} → value replacement via split/join (no recursion)

Section joining

```
Output: Single markdown string  →  # Identity\n---\n# Reasoning Protocols\n---\n# Output Format\n---\n# Task
```

Pipeline Chaining: Multi-Stage Workflows

Templates connected by input/output contracts

How contracts work:

1. Template declares — `output_contract: { type: requirements-document }`
2. Next template declares — `input_contract: { type: requirements-document }`
3. Manifest defines pipeline — `stages: [{produces: ...}, {consumes: ..., produces: ...}]`
4. Bootstrap validates — Previous output matches next input before offering template
5. Artifacts flow forward — Output of stage N becomes input parameter of stage N+1

Testing: Reference Comparison Methodology

How do you test prompts? Compare against known-good references.

Hand-craft reference prompt → **Generate via PromptKit** → **Structured gap analysis** → **Feed gaps back as improvements**

5 Gap Analysis Dimensions

Task Framing

Goal statement, success criteria,
non-goals, context definition

Is the objective clearly stated?
Is scope explicitly bounded?

Reasoning Methodology

Systematic analysis, hypothesis generation, evidence requirements

Is anti-hallucination present?
Are multiple hypotheses required?

Output Specification

Format structure, artifacts, taxonomy, severity ranking

Is the deliverable structure defined?
Are classification schemes provided?

Operational Guidance

Scoping strategy, tool usage, step-by-step plan

Does it guide how to scope work?
Is a procedural plan provided?

Quality Assurance

Self-verification, sampling, coverage, consistency

Must LLM verify its output?
Must it document what it examined?

Get Started Today

Try it now
`npx @alan-jowett/promptkit`

Zero install — walks you through template selection and prompt assembly.

Browse the repo
`github.com/microsoft/promptkit`

Read the templates, protocols, personas. Understand the composition model.

Contribute
`Use the extend-library template`

PromptKit eats its own dog food — the contribution workflow is a PromptKit template.

Share feedback
`File issues or start discussions`

What templates are missing? What workflows do you wish PromptKit supported?